

High-Order Implicit Manufactured FDA Solver Verification Report

Generated report for `highOrderManufacturedFDAImplicit`

May 8, 2026

Abstract

This report documents the current `highOrderManufacturedFDAImplicit` solver. The solver advances the manufactured FDA activation equation coupled to local state variables. The transmembrane potential can be solved with standard cell-centred finite volumes or with an LRE high-order matrix discretization. The nonlinear ionic current can be evaluated at cell centres or volume-integrated from dedicated cell quadrature points, where the local ODE states are also evolved. In contrast with the explicit solver, the potential equation is advanced by an implicit theta method with either backward Euler or Crank–Nicolson time integration, and the high-order potential solve can use either lumped or consistent mass matrices. The tables and plotting script read the current data in `results/example0/2D`.

1 Manufactured FDA Problem

The unknowns are the transmembrane potential V_m and state variables u_1 , u_2 , and u_3 . The manufactured problem is

$$\frac{\partial V_m}{\partial t} = \frac{1}{\chi C_m} \nabla \cdot (D \nabla V_m) - I_{\text{ion}}(V_m, u_1, u_2, u_3), \quad (1)$$

$$\frac{\partial u_1}{\partial t} = (u_1 + u_3 - V_m)^2 u_2^2 + \frac{1}{2} (u_1 + u_3 - V_m) u_2^2 (V_m - u_3), \quad (2)$$

$$\frac{\partial u_2}{\partial t} = -(u_1 + u_3 - V_m) u_2^3, \quad (3)$$

$$\frac{\partial u_3}{\partial t} = 0. \quad (4)$$

The ionic current is

$$I_{\text{ion}} = -\frac{1}{2} (u_1 + u_3 - V_m) u_2^2 (V_m - u_3) + \frac{\beta}{\chi C_m} (V_m - u_3), \quad (5)$$

where β is the exact Laplacian eigenvalue associated with the selected manufactured mode and conductivity tensor. For the two-dimensional runs currently present,

$$F(x, y) = \cos(\pi x) \cos(2\pi y), \quad G(x, y) = 1 + xy^2, \quad (6)$$

$$V_m^*(x, y, t) = \sqrt{1+t} F(x, y), \quad u_1^*(x, y, t) = (1+t)G(x, y) + V_m^*(x, y, t), \quad (7)$$

$$u_2^*(x, y, t) = \frac{1}{(1+t)\sqrt{G(x, y)}}, \quad u_3^* = 0. \quad (8)$$

2 Spatial Discretization

2.1 Potential Equation

For `useHighOrder_Vm=false`, the solver uses the standard OpenFOAM cell-centred finite-volume Laplacian. For `useHighOrder_Vm=true`, the semi-discrete potential equation is assem-

bled in matrix form using LRE reconstructions,

$$M \frac{dV}{dt} = KV - MI_{\text{ion}} + f, \quad (9)$$

where M is the selected mass matrix, K is the high-order stiffness/diffusion matrix, and f contains manufactured and boundary contributions. The mass matrix can be **lumped**, giving a diagonal cell-volume-like mass, or **consistent**, obtained by integrating the reconstruction basis over cell quadrature points. On purely standard runs, the consistent/lumped distinction collapses to the usual diagonal finite-volume mass.

2.2 Volume Integration of I_{ion}

The current solver uses `useHighOrder_Iion` for the nonlinear source. The older input name `useHighOrder_states`, and the result-folder token `hoStates`, are still accepted for compatibility, but should be interpreted as the quadrature level used for I_{ion} and for the local state values stored at those quadrature points. If high-order ionic-current quadrature is inactive,

$$I_{\text{ion},K}^n = I_{\text{ion}}(V_{m,K}^n, u_{1,K}^n, u_{2,K}^n, u_{3,K}^n). \quad (10)$$

If it is active, the source is volume-integrated as

$$I_{\text{ion},K}^n \approx \frac{1}{|K|} \sum_{q=1}^{N_K} \omega_{Kq} I_{\text{ion}}(V_m^n(\mathbf{x}_{Kq}), u_1^n(\mathbf{x}_{Kq}), u_2^n(\mathbf{x}_{Kq}), u_3^n(\mathbf{x}_{Kq})). \quad (11)$$

The value $V_m(\mathbf{x}_{Kq})$ is reconstructed with the V_m LRE operator when high-order V_m is active, and otherwise by a linear cell-centred reconstruction.

3 Boundary Conditions

For the standard finite-volume operator, the OpenFOAM patch fields are used directly. For the high-order diffusive flux, `zeroGradient` and `empty` patches are treated as no-flux patches in the assembled high-order operator:

$$\int_{\Gamma_N \cap \partial K} \mathbf{n} \cdot D \nabla V_m dS = 0. \quad (12)$$

Fixed-value patches keep their boundary contribution in the high-order matrix/source assembly.

4 Implicit Time Integration and Nonlinear Treatment

The solver advances the potential equation with a theta method,

$$\theta = 1 \quad \text{for backward Euler (bE)}, \quad \theta = \frac{1}{2} \quad \text{for Crank–Nicolson (cN)}. \quad (13)$$

After spatial discretization, the potential equation can be written as

$$M \frac{V^{n+1} - V^n}{\Delta t} = L (\theta V^{n+1} + (1 - \theta) V^n) + \theta S^{n+1} + (1 - \theta) S^n, \quad (14)$$

where V contains the cell-centred unknowns, M is the selected mass matrix, L is the scaled diffusion/reaction matrix assembled from the high-order or standard spatial operator, and

$$S(V, u_1, u_2, u_3) = -I_{\text{ion}}(V, u_1, u_2, u_3) + f_{\text{MMS}}. \quad (15)$$

The linear system solved in each nonlinear correction is therefore

$$\left(\frac{M}{\Delta t} - \theta L \right) V^{k+1} = \left(\frac{M}{\Delta t} + (1 - \theta) L \right) V^n + \theta S(V^k, u_1^k, u_2^k, u_3^k) + (1 - \theta) S(V^n, u_1^n, u_2^n, u_3^n), \quad (16)$$

where k denotes the outer nonlinear correction index inside one time step.

4.1 Backward Euler and Crank–Nicolson

Backward Euler is first-order accurate in time. Its main advantage is strong numerical damping and robust behaviour for stiff problems. The price is larger time-discretization error and stronger damping of fast transients, so activation or reaction-diffusion fronts may be smeared if Δt is too large. Crank–Nicolson is second-order accurate for smooth solutions and is less dissipative. This is advantageous for manufactured-solution convergence studies, but the weaker damping can make nonlinear or front-like problems more sensitive to oscillations, splitting error, and insufficient nonlinear convergence. In this solver cN uses the same nonlinear correction machinery as bE, so its formal second-order behaviour is best expected when the source and state updates are also converged sufficiently.

4.2 Lumped and Consistent Mass Matrices

For the lumped option, the mass matrix is diagonal,

$$M_{ii}^{\text{lump}} \approx |K_i|, \quad (17)$$

with one degree of freedom per control volume. This is cheap, positive, and well-conditioned, and it coincides with the usual cell-centred finite-volume mass. It is also the fallback used when a standard, non-high-order potential solve is requested.

For the consistent option, the mass matrix is assembled using the same LRE reconstruction/quadrature framework as the high-order spatial operator. In abstract form,

$$M_{ij}^{\text{cons}} \approx \sum_K \sum_{q \in K} \omega_{Kq} \phi_i(\mathbf{x}_{Kq}) \phi_j(\mathbf{x}_{Kq}), \quad (18)$$

where ϕ_i are the reconstructed basis functions induced by the LRE stencil and ω_{Kq} are the cell quadrature weights. This is a more faithful mass operator for high-order reconstructions, but it is sparse rather than diagonal, more expensive to assemble and solve with, and can be more sensitive to stencil quality and conditioning. In the current implementation, requesting a consistent mass without a high-order V_m reconstruction reduces to the diagonal/lumped finite-volume mass because there is no high-order basis to integrate.

4.3 Treatment of the Nonlinear Ionic Current

The nonlinear current depends on both the potential and the local states,

$$I_{\text{ion}} = I_{\text{ion}}(V_m, u_1, u_2, u_3). \quad (19)$$

The current implementation does not assemble a Newton Jacobian for this term. Instead, it uses two nested fixed-point mechanisms that are actually active in `highOrderManufacturedFDAImplicit`. The outer correction is controlled by `implicitNonlinearIterations`, which is read from `implicitHOCoeffs/nonlinearIterations`. In the tutorial inputs used for the manufactured implicit runs this value is currently

```
implicitNonlinearIterations = 3
```

Therefore, each time step performs three Picard-type corrections of the coupled potential/source system.

At the beginning of a time step the solver stores V^n and the old states. It then computes S^n from the old fields. For each outer correction k , it evaluates the source using the current guess $(V^k, u_1^k, u_2^k, u_3^k)$, solves Eq. (16) for a new potential, and then updates the local state variables with the same theta choice. This outer loop corresponds to the code block

```
for corr < max(implicitNonlinearIterations, 1)
```

The state update has its own local fixed-point loop controlled by `stateImplicitIterations`, read from `implicitHOCoeffs/stateIterations`. For the current manufactured implicit tutorial this value is

$$\text{stateImplicitIterations} = 5$$

Thus, inside each outer potential/source correction, the local ODE update for u_1 , u_2 , and u_3 is swept five times. This is used both for the cell-centred update `updateStateFieldsTheta` and for the quadrature-point update `updateIntegrationPointStatesTheta`. In both cases the repeated map has the form

$$u^{m+1} = u^n + \Delta t \left[(1 - \theta)R(V^n, u^n) + \theta R(V^{k+1}, u^m) \right], \quad (20)$$

where m is the inner state fixed-point iteration.

If `implicitNonlinearIterations=1`, the method is best interpreted as semi-implicit: the diffusion operator is implicit, but the nonlinear source is evaluated from a lagged or once-updated guess. With the current value `implicitNonlinearIterations=3`, the implementation is a Picard/fixed-point coupling between V_m and the nonlinear source. It is still not Newton because the matrix in Eq. (16) does not include derivatives such as $\partial I_{\text{ion}}/\partial V_m$ or $\partial I_{\text{ion}}/\partial u_i$, and the present code does not check a nonlinear residual nor stop on a convergence tolerance.

When `useHighOrder_Iion=false`, the nonlinear source and states are updated at cell centres. When `useHighOrder_Iion=true`, the states are stored and advanced at the I_{ion} quadrature points, V_m is reconstructed to those points, and the cell source is obtained by quadrature averaging. Thus the “hoStates” result-folder token should be read as the high-order quadrature level used for the ionic current and associated state evolution.

4.4 Algorithm per Time Step

Algorithm 1 summarizes one time step. The notation u denotes the state vector (u_1, u_2, u_3) , and S denotes the manufactured source contribution containing the nonlinear ionic current.

4.5 Toward a Fully Implicit Nonlinear Solve

The Picard treatment is simple and avoids differentiating the ionic model. It is therefore robust to implement and keeps the linear system matrix independent of the current nonlinear guess. Its disadvantages are that the nonlinear source is only weakly coupled to the potential, convergence can be slow, and with only one correction the method retains an explicit/lagged character in the source term. Several stronger alternatives are possible:

- (i) **Residual-controlled Picard:** keep the current fixed-point structure, but iterate until $\|V^{k+1} - V^k\|$ or the nonlinear residual is below a tolerance. This is the least invasive improvement.
- (ii) **Newton with an approximate ionic Jacobian:** add $\partial I_{\text{ion}}/\partial V_m$ to the implicit matrix while still updating the states separately. This gives a more implicit voltage-source coupling without requiring the full state Jacobian.
- (iii) **Fully coupled Newton:** solve simultaneously for (V_m, u_1, u_2, u_3) at cells or quadrature points. This is the most consistent formulation, but it requires the full Jacobian of the PDE–ODE residual and a larger block system.
- (iv) **Jacobian-free Newton–Krylov:** compute nonlinear residuals and use finite-difference Jacobian-vector products. This avoids deriving all analytic partials, but requires robust preconditioning and careful scaling of the PDE and ODE unknowns.

Algorithm 1 One implicit time step in `highOrderManufacturedFDAImplicit`

- 1: Store old fields V^n , u^n and initialize guesses $V^0 \leftarrow V^n$, $u^0 \leftarrow u^n$.
- 2: Compute the old source $S^n = S(V^n, u^n)$.
- 3: **if** `useHighOrder_Iion=true` **then**
- 4: Reconstruct V^n to the I_{ion} quadrature points.
- 5: Evaluate I_{ion} and the manufactured source at those quadrature points.
- 6: Average the quadrature source back to cell centres.
- 7: **else**
- 8: Evaluate I_{ion} and the manufactured source directly at cell centres.
- 9: **end if**
- 10: Let N_{nl} denote `implicitNonlinearIterations`.
- 11: **for** $k = 0$ to $\max(N_{\text{nl}}, 1) - 1$ **do**
- 12: Evaluate the guessed source $S^k = S(V^k, u^k)$, using cell-centred or quadrature-point evaluation according to `useHighOrder_Iion`.
- 13: Assemble the right-hand side

$$b^k = \left(\frac{M}{\Delta t} + (1 - \theta)L \right) V^n + \theta S^k + (1 - \theta)S^n.$$

- 14: Solve the linear potential system

$$\left(\frac{M}{\Delta t} - \theta L \right) V^{k+1} = b^k.$$

- 15: Update the state variables with the same theta choice, using V^n and V^{k+1} ; repeat the local state fixed-point map `stateImplicitIterations` times.
 - 16: Apply/update boundary conditions for V^{k+1} and u^{k+1} .
 - 17: **end for**
 - 18: Accept the last correction: $V^{n+1} \leftarrow V^{k+1}$ and $u^{n+1} \leftarrow u^{k+1}$.
 - 19: Write diagnostics and advance the OpenFOAM time object.
-

A fully implicit solver would target the residual

$$R(V^{n+1}, u^{n+1}) = 0, \tag{21}$$

with both the diffusion and the nonlinear ionic current evaluated at the new time level. That approach is more expensive per step, but it is the cleanest way to remove the lagged-source error when large time steps or strongly nonlinear front dynamics are important.

5 Error Measures and Convergence Rates

The solver writes cell-centred volume-weighted L_1 , L_2 , and L_∞ errors for V_m , u_1 , and u_2 . It also writes reconstructed quadrature errors, denoted by the superscript G . Pairwise rates use consecutive mesh sizes,

$$p_{N_1, N_2} = \frac{\log(E_{N_1}/E_{N_2})}{\log(h_{N_1}/h_{N_2})}, \quad h_N = 1/N. \tag{22}$$

The convergence tables below recompute log-log fits directly from the `errors\_vs\_N` files, using both the full $N = 10$ – 100 mesh range and the finer $N = 70$ – 100 range.

6 Observed Orders

The convergence orders in the following tables are computed directly from the current `errors\_vs\_N` files. For each available time step, one table is reported. The first six order columns use a log-log fit over meshes $N = 10$ – 100 , and the final six order columns use the finest range $N = 70$ – 100 .

Table 1: Implicit solver, 2D results for $\Delta t = 10^{-3}$. The first six convergence columns are log-log fits over meshes $N = 10$ – 100 ; the last six columns are fits over the finest range $N = 70$ – 100 . Here bE denotes backward Euler, cN denotes Crank–Nicolson, cons. denotes a consistent mass matrix, and lump. denotes a lumped mass matrix.

V_m	I_{ion}	Scheme	Mass	Fit $N = 10$ – 100						Fit $N = 70$ – 100					
				V_m	V_m^G	u_1	u_1^G	u_2	u_2^G	V_m	V_m^G	u_1	u_1^G	u_2	u_2^G
N0	N0	bE	cons.	2.173	2.173	0.682	0.682	0.008	0.008	2.657	2.657	0.116	0.116	0.000	0.000
N0	p1	bE	cons.	2.171	2.171	1.980	1.793	0.499	0.469	2.650	2.650	1.937	1.192	0.056	0.019
p1	N0	bE	cons.	1.914	2.150	1.540	1.540	0.296	0.296	2.618	2.180	1.864	1.864	0.022	0.022
p1	p1	bE	cons.	1.914	2.150	1.979	2.060	0.597	0.872	2.618	2.180	1.943	1.877	0.078	0.049
p2	N0	bE	cons.	2.232	2.699	1.356	1.356	0.115	0.115	2.209	2.421	0.363	0.363	0.000	0.000
p2	p1	bE	cons.	2.245	2.716	1.983	1.968	0.382	0.430	2.219	2.442	1.933	0.432	0.040	0.000
p2	p2	bE	cons.	2.275	2.753	1.993	2.253	0.327	0.615	2.242	2.491	1.965	0.536	-0.046	0.001
p3	N0	bE	cons.	3.179	3.660	0.580	0.580	0.007	0.007	6.392	5.748	0.042	0.042	-0.000	-0.000
p3	p1	bE	cons.	3.077	3.678	1.980	0.994	0.308	0.062	2.207	3.046	1.932	0.031	0.039	-0.000
p3	p3	bE	cons.	2.572	3.345	1.989	1.308	0.153	0.142	0.176	0.267	1.964	0.010	-0.047	-0.000
N0	N0	bE	lump.	2.173	2.173	0.682	0.682	0.008	0.008	2.657	2.657	0.116	0.116	0.000	0.000
N0	p1	bE	lump.	2.171	2.171	1.980	1.793	0.499	0.469	2.650	2.650	1.937	1.192	0.056	0.019
p1	N0	bE	lump.	1.914	2.150	1.540	1.540	0.296	0.296	2.618	2.180	1.864	1.864	0.022	0.022
p1	p1	bE	lump.	1.914	2.150	1.979	2.060	0.597	0.872	2.618	2.180	1.943	1.877	0.078	0.049
p2	N0	bE	lump.	2.184	2.646	1.368	1.368	0.118	0.118	2.182	2.363	0.405	0.405	0.000	0.000
p2	p1	bE	lump.	2.195	2.662	1.983	1.959	0.383	0.431	2.189	2.380	1.933	0.469	0.040	0.001
p2	p2	bE	lump.	2.218	2.696	1.992	2.242	0.328	0.615	2.207	2.419	1.965	0.566	-0.046	0.001
p3	N0	bE	lump.	2.447	2.921	0.688	0.688	0.011	0.011	3.063	3.108	0.084	0.084	0.000	0.000
p3	p1	bE	lump.	2.546	3.055	1.980	1.026	0.310	0.064	3.326	3.374	1.932	0.072	0.039	-0.000
p3	p3	bE	lump.	3.132	3.636	1.988	1.316	0.157	0.143	6.234	5.651	1.965	0.051	-0.047	-0.000
N0	N0	cN	cons.	1.997	1.997	1.997	1.997	1.993	1.993	2.000	2.000	2.001	2.001	1.976	1.976
N0	p1	cN	cons.	1.997	1.997	1.999	2.050	2.000	2.054	2.000	2.000	2.000	2.007	2.005	2.008
p1	N0	cN	cons.	1.912	2.147	1.616	1.616	1.626	1.626	2.611	2.168	2.380	2.380	2.385	2.385
p1	p1	cN	cons.	1.912	2.147	1.997	2.152	1.837	2.158	2.611	2.168	2.005	2.308	2.251	2.308
p2	N0	cN	cons.	2.179	2.658	2.197	2.197	2.197	2.197	2.027	2.256	2.033	2.033	2.031	2.031
p2	p1	cN	cons.	2.190	2.674	2.003	2.748	2.110	2.752	2.028	2.272	2.001	2.374	2.024	2.380
p2	p2	cN	cons.	2.214	2.709	2.003	2.924	2.111	2.923	2.032	2.308	2.000	2.644	1.998	2.647
p3	N0	cN	cons.	2.446	3.036	2.465	2.465	2.440	2.440	2.075	2.211	2.140	2.140	1.974	1.974
p3	p1	cN	cons.	2.593	3.213	2.001	3.174	1.993	3.117	2.127	2.319	2.000	2.353	2.016	2.028
p3	p3	cN	cons.	3.538	3.993	1.999	3.998	1.995	3.835	4.323	4.279	2.000	4.182	1.990	2.652
N0	N0	cN	lump.	1.997	1.997	1.997	1.997	1.993	1.993	2.000	2.000	2.001	2.001	1.976	1.976
N0	p1	cN	lump.	1.997	1.997	1.999	2.050	2.000	2.054	2.000	2.000	2.000	2.007	2.005	2.008
p1	N0	cN	lump.	1.912	2.147	1.616	1.616	1.626	1.626	2.611	2.168	2.380	2.380	2.385	2.385
p1	p1	cN	lump.	1.912	2.147	1.997	2.152	1.837	2.158	2.611	2.168	2.005	2.308	2.251	2.308
p2	N0	cN	lump.	2.137	2.608	2.153	2.153	2.153	2.153	2.021	2.214	2.026	2.026	2.024	2.024
p2	p1	cN	lump.	2.146	2.623	2.003	2.702	2.101	2.706	2.023	2.226	2.000	2.319	2.023	2.325
p2	p2	cN	lump.	2.165	2.654	2.003	2.893	2.106	2.892	2.025	2.255	2.000	2.580	1.998	2.584
p3	N0	cN	lump.	2.187	2.667	2.200	2.200	2.195	2.195	2.021	2.081	2.038	2.038	1.993	1.993
p3	p1	cN	lump.	2.232	2.751	2.001	2.716	1.993	2.702	2.028	2.101	2.000	2.100	2.016	2.029
p3	p3	cN	lump.	2.418	3.024	1.999	3.312	1.995	3.282	2.068	2.202	2.000	2.409	1.990	2.256

Table 2: Implicit solver, 2D results for $\Delta t = 10^{-4}$. The first six convergence columns are log-log fits over meshes $N = 10$ – 100 ; the last six columns are fits over the finest range $N = 70$ – 100 . Here bE denotes backward Euler, cN denotes Crank–Nicolson, cons. denotes a consistent mass matrix, and lump. denotes a lumped mass matrix.

V_m	I_{ion}	Scheme	Mass	Fit $N = 10$ – 100						Fit $N = 70$ – 100					
				V_m	V_m^G	u_1	u_1^G	u_2	u_2^G	V_m	V_m^G	u_1	u_1^G	u_2	u_2^G
N0	N0	cN	cons.	1.997	1.997	1.997	1.997	1.997	1.997	2.000	2.000	2.000	2.000	2.000	2.000
N0	p1	cN	cons.	1.997	1.997	1.999	2.050	1.998	2.053	2.000	2.000	2.000	2.007	2.000	2.007
p1	N0	cN	cons.	1.912	2.147	1.616	1.616	1.626	1.626	2.611	2.168	2.380	2.380	2.385	2.385
p1	p1	cN	cons.	1.912	2.147	1.997	2.152	1.836	2.158	2.611	2.168	2.005	2.308	2.249	2.308
p2	N0	cN	cons.	2.179	2.658	2.197	2.197	2.197	2.197	2.027	2.256	2.033	2.033	2.033	2.033
p2	p1	cN	cons.	2.190	2.674	2.003	2.748	2.106	2.752	2.028	2.272	2.000	2.374	2.011	2.383
p2	p2	cN	cons.	2.214	2.709	2.003	2.924	2.114	2.923	2.032	2.309	2.000	2.643	2.007	2.649
p3	N0	cN	cons.	2.447	3.036	2.464	2.464	2.466	2.466	2.075	2.211	2.136	2.136	2.132	2.132
p3	p1	cN	cons.	2.593	3.213	2.001	3.173	1.987	3.168	2.127	2.319	2.000	2.349	1.999	2.334
p3	p3	cN	cons.	3.539	3.993	1.999	3.997	1.998	4.005	4.323	4.280	2.000	4.178	2.000	4.178

7 Convergence Figures

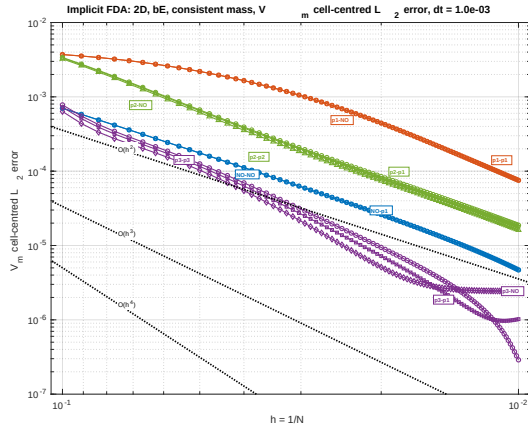
Run `plot_convergence.m` from this report directory to regenerate the PDF and PNG figures. The script scans the available result dimensions and generates individual plot files for each

dimension, time step, implicit time scheme, mass matrix, variable, and error metric. The report assembles these individual files into one six-panel figure for each available $(\Delta t, \text{scheme}, \text{mass})$ combination. In each figure, rows correspond to V_m , u_1 , and u_2 , while the two columns compare cell-centred L_2 and reconstructed quadrature G_2 errors. The legend is placed outside each plot, common axis limits are used across time steps for a fixed scheme/mass/variable/metric family, and the horizontal axis is reversed so refinement proceeds from left to right.

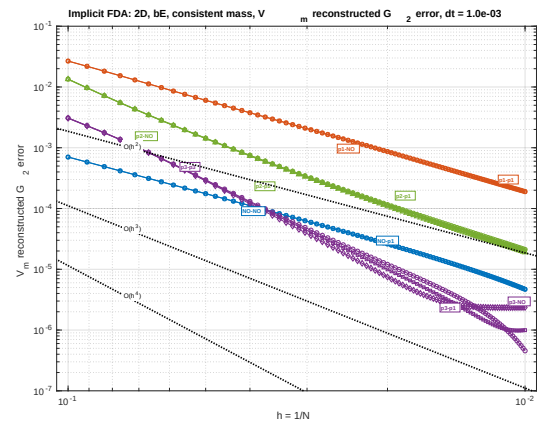
7.1 2D Figures

$$\Delta t = 10^{-3}$$

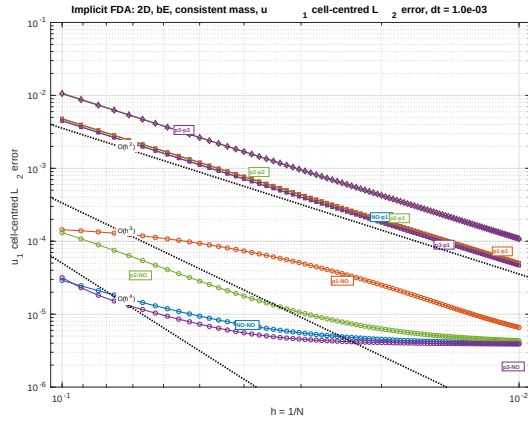
$$\Delta t = 10^{-4}$$



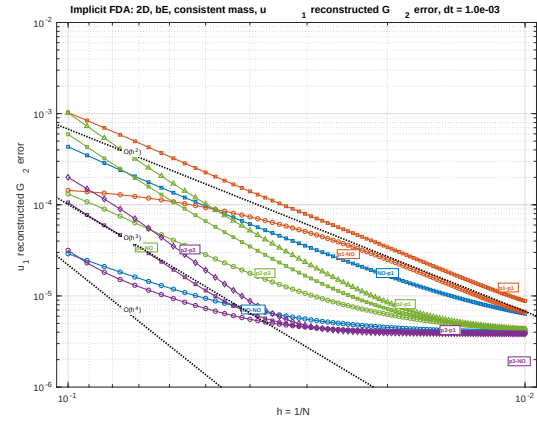
(a) $\|V_m\|_{L_2}$



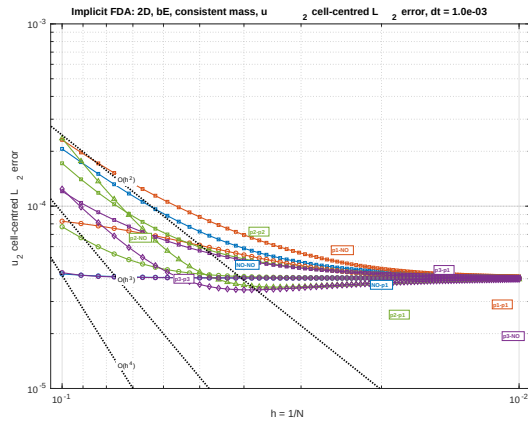
(b) $\|V_m\|_{G_2}$



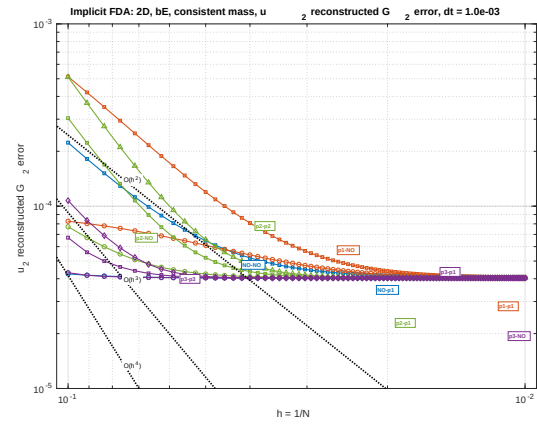
(c) $\|u_1\|_{L_2}$



(d) $\|u_1\|_{G_2}$

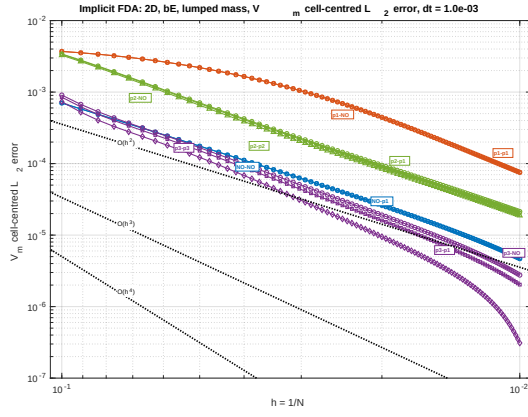


(e) $\|u_2\|_{L_2}$

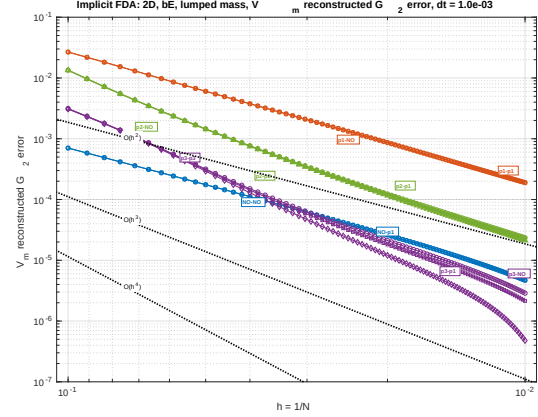


(f) $\|u_2\|_{G_2}$

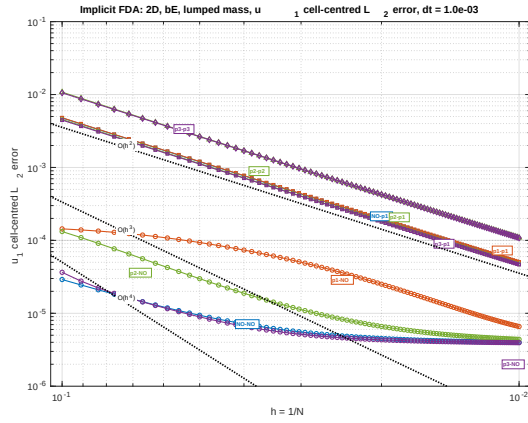
Figure 1: Implicit manufactured FDA solver, 2D, bE with cons. mass, $\Delta t = 10^{-3}$. Each row corresponds to one variable: V_m , u_1 , and u_2 from top to bottom. The left column shows the cell-centred L_2 error and the right column shows the reconstructed quadrature G_2 error.



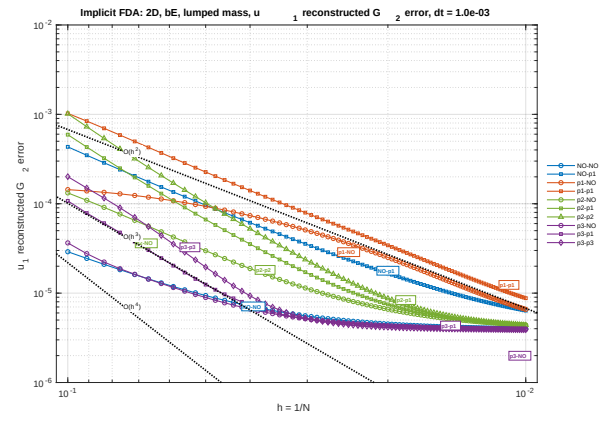
(a) $\|V_m\|_{L_2}$



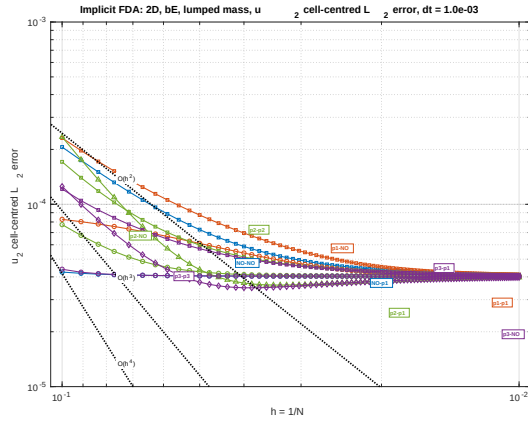
(b) $\|V_m\|_{G_2}$



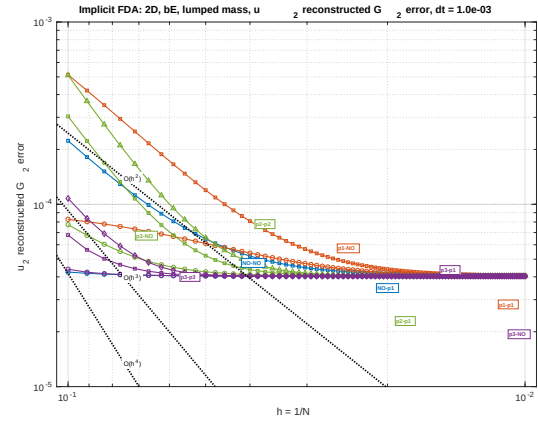
(c) $\|u_1\|_{L_2}$



(d) $\|u_1\|_{G_2}$

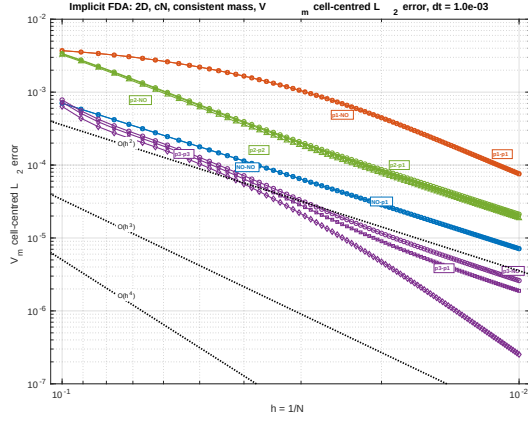


(e) $\|u_2\|_{L_2}$

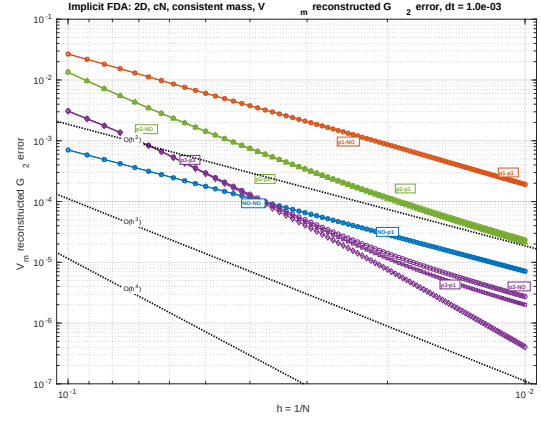


(f) $\|u_2\|_{G_2}$

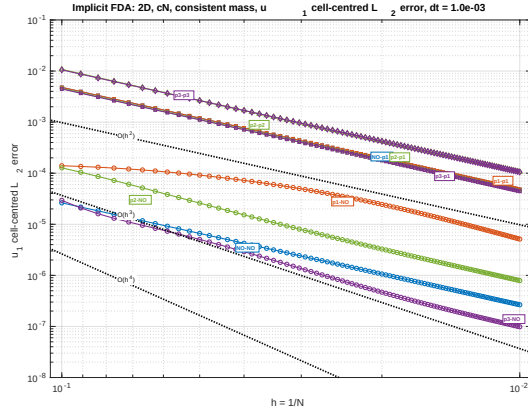
Figure 2: Implicit manufactured FDA solver, 2D, bE with lump. mass, $\Delta t = 10^{-3}$. Each row corresponds to one variable: V_m , u_1 , and u_2 from top to bottom. The left column shows the cell-centred L_2 error and the right column shows the reconstructed quadrature G_2 error.



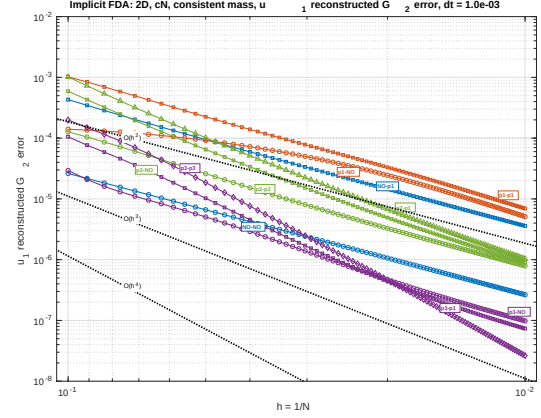
(a) $\|V_m\|_{L_2}$



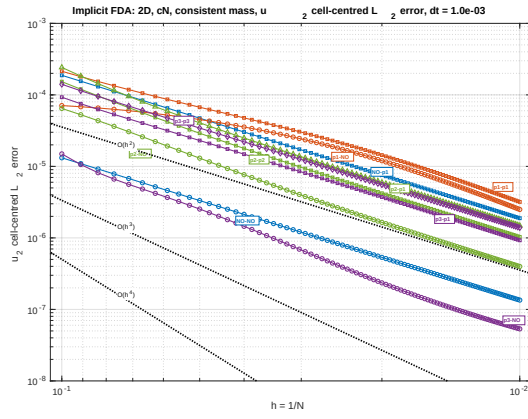
(b) $\|V_m\|_{G_2}$



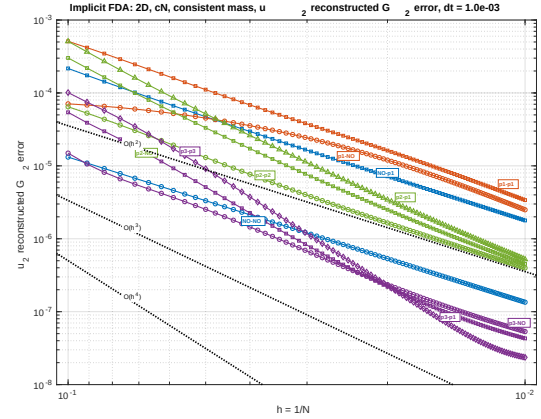
(c) $\|u_1\|_{L_2}$



(d) $\|u_1\|_{G_2}$

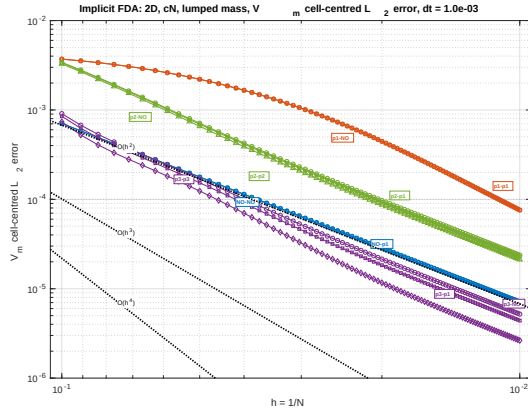


(e) $\|u_2\|_{L_2}$

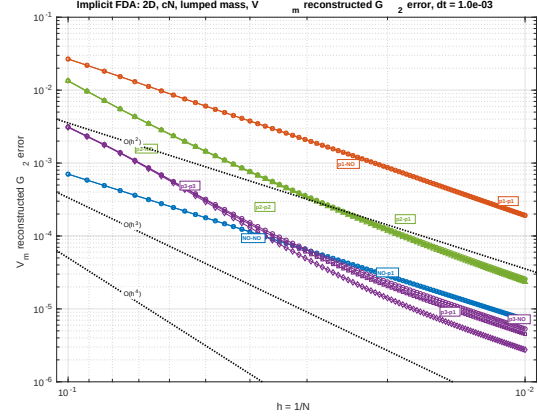


(f) $\|u_2\|_{G_2}$

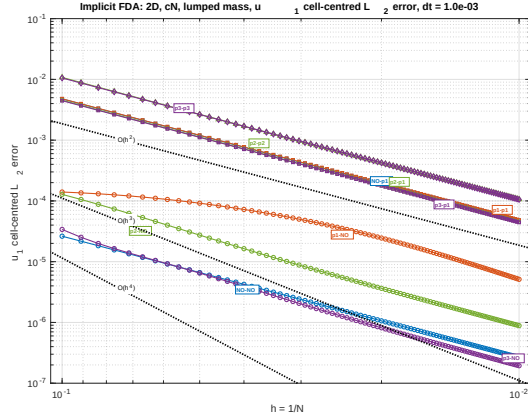
Figure 3: Implicit manufactured FDA solver, 2D, cN with cons. mass, $\Delta t = 10^{-3}$. Each row corresponds to one variable: V_m , u_1 , and u_2 from top to bottom. The left column shows the cell-centred L_2 error and the right column shows the reconstructed quadrature G_2 error.



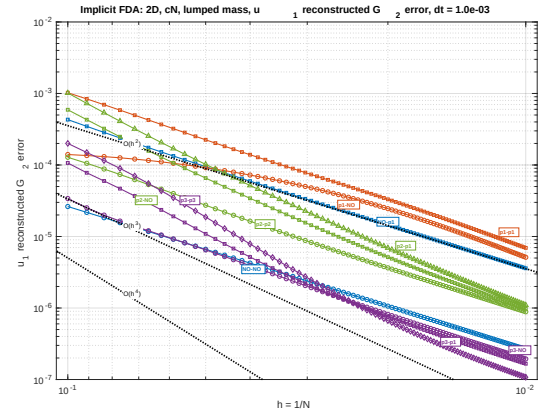
(a) $\|V_m\|_{L_2}$



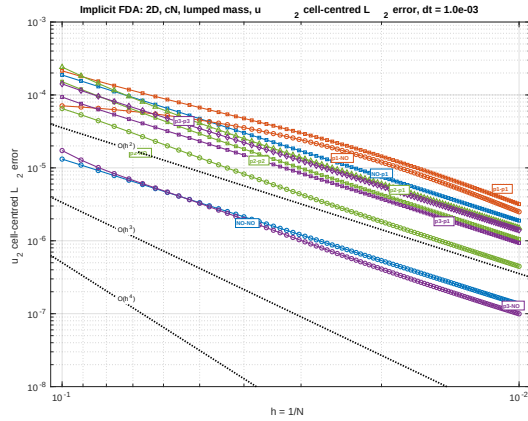
(b) $\|V_m\|_{G_2}$



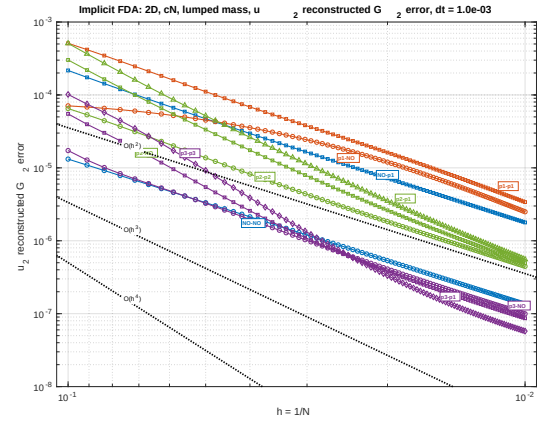
(c) $\|u_1\|_{L_2}$



(d) $\|u_1\|_{G_2}$

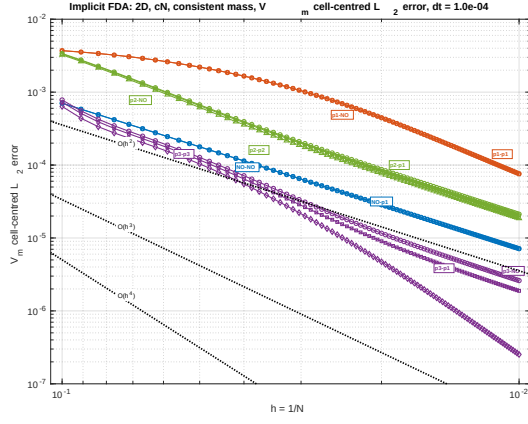


(e) $\|u_2\|_{L_2}$

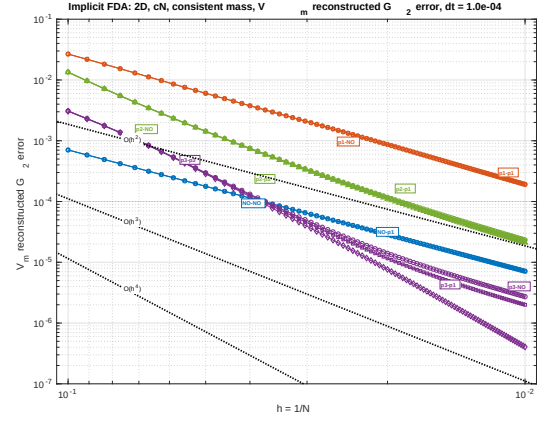


(f) $\|u_2\|_{G_2}$

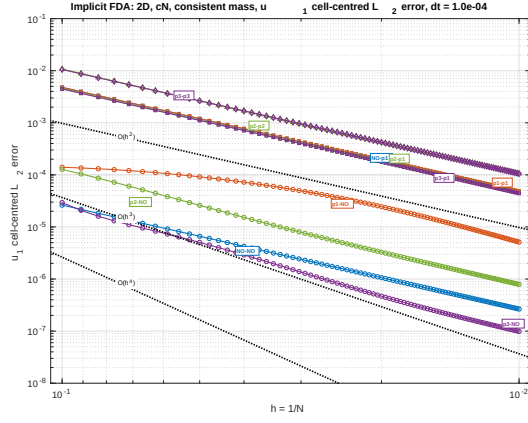
Figure 4: Implicit manufactured FDA solver, 2D, cN with lump. mass, $\Delta t = 10^{-3}$. Each row corresponds to one variable: V_m , u_1 , and u_2 from top to bottom. The left column shows the cell-centred L_2 error and the right column shows the reconstructed quadrature G_2 error.



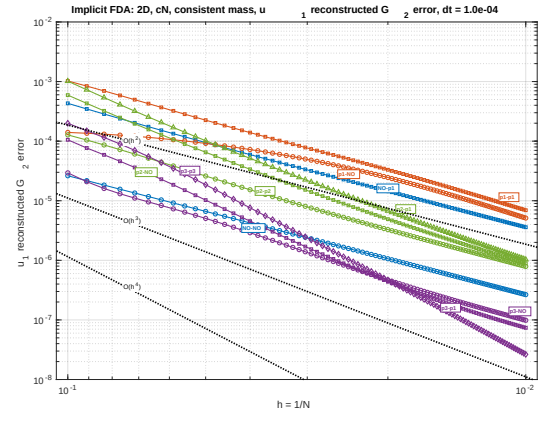
(a) $\|V_m\|_{L_2}$



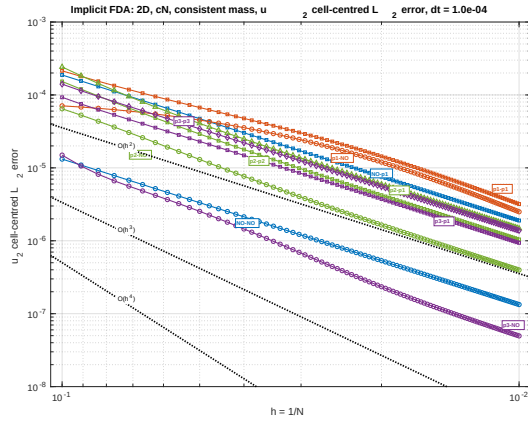
(b) $\|V_m\|_{G_2}$



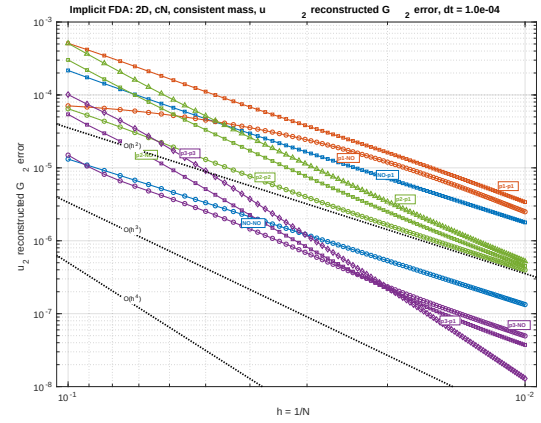
(c) $\|u_1\|_{L_2}$



(d) $\|u_1\|_{G_2}$



(e) $\|u_2\|_{L_2}$



(f) $\|u_2\|_{G_2}$

Figure 5: Implicit manufactured FDA solver, 2D, cN with cons. mass, $\Delta t = 10^{-4}$. Each row corresponds to one variable: V_m , u_1 , and u_2 from top to bottom. The left column shows the cell-centred L_2 error and the right column shows the reconstructed quadrature G_2 error.

8 Notes

Backward Euler is first-order in time and Crank–Nicolson is second-order for the smooth manufactured problem. Comparing the same spatial configuration across the bE/cN figures indicates whether the spatial error or time discretization dominates. The mass comparison isolates the effect of replacing the diagonal/lumped finite-volume mass by the consistent high-order mass matrix in the implicit potential solve. The reconstructed quantities V_m^G , u_1^G , and u_2^G should be read as quadrature-point/reconstructed diagnostics; the cell-centred fields are the values used in the main finite-volume update and in the L_2 convergence table.